

```
[2]: # Pandas is a software library written for the Python programming language for data manipulation and analysis.
!pip install pandas
import pandas as pd
# NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection
!pip install numpy
import numpy as np
# Matplotlib is a plotting library for python and pyplot gives us a MatLab like plotting framework. We will use this in our plotter function to plot data
!pip install matplotlib
import matplotlib.pyplot as plt
# Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical plots
!pip install seaborn
import seaborn as sns
# Preprocessing allows us to standardize our data
from sklearn import preprocessing
# Allows us to split our data into training and testing data
from sklearn.model_selection import train_test_split
# Allows us to test parameters of classification algorithms and find the best one
from sklearn.model_selection import GridSearchCV
# Logistic Regression classification algorithm
from sklearn.linear_model import LogisticRegression
# Support Vector Machine classification algorithm
from sklearn.svm import SVC
# Decision Tree classification algorithm
from sklearn.tree import DecisionTreeClassifier
# K Nearest Neighbors classification algorithm
from sklearn.neighbors import KNeighborsClassifier
```

```
[19]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, train_test_split

# 1. Define the plot function
def plot_confusion_matrix(y, y_predict):
    "this function plots the confusion matrix"
    from sklearn.metrics import confusion_matrix

    cm = confusion_matrix(y, y_predict)
    ax = plt.subplot()
    sns.heatmap(cm, annot=True, ax=ax) # annot=True to annotate cells
    ax.set_xlabel('Predicted labels')
    ax.set_ylabel('True labels')
    ax.set_title('Confusion Matrix')
    ax.xaxis.set_ticklabels(['did not land', 'land'])
    ax.yaxis.set_ticklabels(['did not land', 'landed'])
    plt.show()

from sklearn import preprocessing

# 1. Initialize the StandardScaler object
transform = preprocessing.StandardScaler()
transform = preprocessing.StandardScaler()
# 2. Extract targets and scale features
import pandas as pd
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# 1. Load the dataset for X (or Load from URL)
X = pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_3.csv")

# 2. Initialize scaler and target variable
transform = preprocessing.StandardScaler()
Y = data['Class'].to_numpy()

# 3. Fit transform X
```

```
# 4. Perform train/test split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
Y = data['Class'].to_numpy()
X = transform.fit_transform(X)

# 3. Perform train/test split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
# 2. Extract targets and scale features
Y = data['Class'].to_numpy()
X = transform.fit_transform(X)

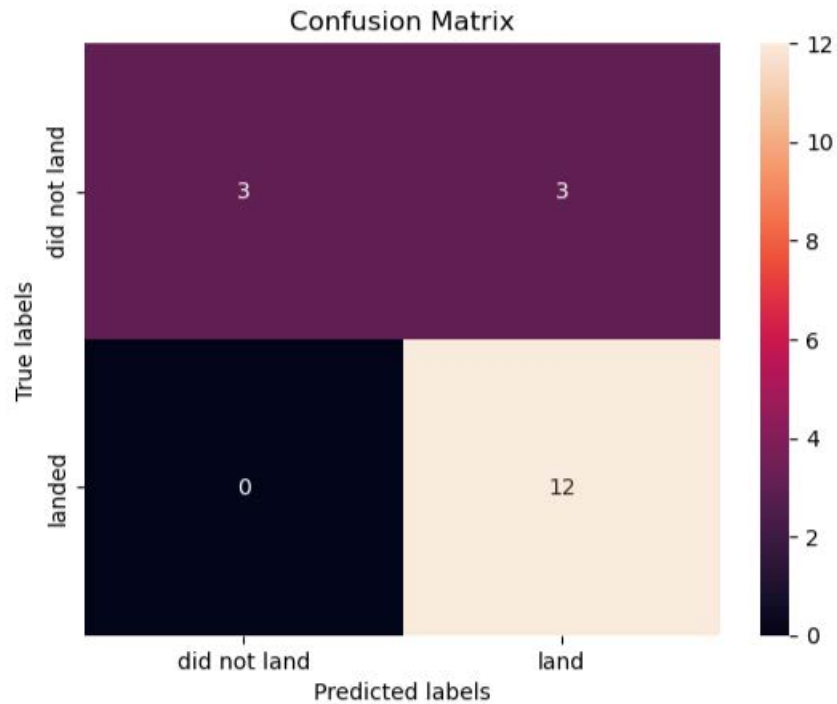
# 3. Perform train/test split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)

# 4. Define parameters and fit model
parameters = {'C': [0.01, 0.1, 1],
              'penalty': ['l2'],
              'solver': ['lbfgs']}

lr = LogisticRegression()
logreg_cv = GridSearchCV(lr, parameters, cv=10)
logreg_cv.fit(X_train, Y_train)

# 5. Make predictions and display matrix
yhat = logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test, yhat)
```

```
plot_confusion_matrix(Y_test, yhat)
```



▼ Load the dataframe

Load the data

```
[8]: URL1 = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_2.csv"
data = pd.read_csv(URL1)
```

```
[9]: data.head()
```

```
[9]:
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Long
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.5°
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.5°
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.5°
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.6°
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.5°

```
[23]: URL2 = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_3.csv'
X = pd.read_csv(URL2)
```

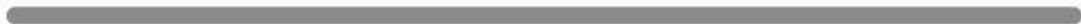
```
[23]: URL2 = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_3.csv'
X = pd.read_csv(URL2)
```

```
[24]: X.head(100)
```

```
[24]:
```

	FlightNumber	PayloadMass	Flights	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	Orbit_GTO	Orbit_HEO	Orbit_ISS	...	Serial_B1058	Serial_B1059	Serial_B1060	S
0	1.0	6104.959412	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
1	2.0	525.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
2	3.0	677.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...	0.0	0.0	0.0	
3	4.0	500.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
4	5.0	3170.000000	1.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	...	0.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
85	86.0	15400.000000	2.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	1.0	
86	87.0	15400.000000	3.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	0.0	
87	88.0	15400.000000	6.0	5.0	5.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
88	89.0	15400.000000	3.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	1.0	
89	90.0	3681.000000	1.0	5.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	

90 rows × 83 columns



TASK 1

Create a NumPy array from the column `Class` in `data`, by applying the method `to_numpy()` then assign it to the variable `Y`, make sure the output is a Pandas series (only one bracket df['name of column']).

```
[25]: Y = data['Class'].to_numpy()
```

TASK 2

Standardize the data in `X` then reassign it to the variable `X` using the transform provided below.

```
[ ]: # students get this
transform = preprocessing.StandardScaler()
X = transform.fit_transform(X)
```

We split the data into training and testing data using the function `train_test_split`. The training data is divided into validation data, a second set used for training data; then the models are trained and hyperparameters are selected using the function `GridSearchCV`.

TASK 3

Use the function `train_test_split` to split the data `X` and `Y` into training and test data. Set the parameter `test_size` to 0.2 and `random_state` to 2. The training data and test data should be assigned to the following labels.

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
```

```
[ ]:
```

we can see we only have 18 test samples.

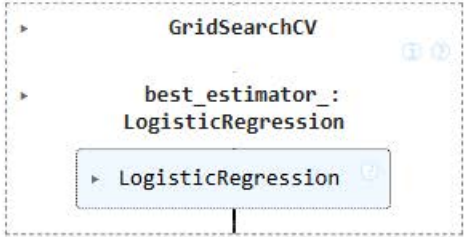
```
[27]: Y_test.shape
```

TASK 4

Create a logistic regression object then create a GridSearchCV object `logreg_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
[28]: parameters = {'C':[0.01,0.1,1],
                  'penalty':['l2'],
                  'solver':['lbfgs']}
lr = LogisticRegression()
logreg_cv = GridSearchCV(lr, parameters, cv=10)
logreg_cv.fit(X_train, Y_train)
```

```
[28]:
```



```
GridSearchCV
├── best_estimator_:
│   └── LogisticRegression
```

```
[29]: parameters ={"C":[0.01,0.1,1], 'penalty':['l2'], 'solver':['lbfgs']}# l1 Lasso l2 ridge
lr=LogisticRegression()
```

We output the `GridSearchCV` object for logistic regression. We display the best parameters using the data attribute `best_params_` and the accuracy on the validation data using the data attribute `best_score_`.

```
[30]: print("tuned hyperparameters :(best parameters) ",logreg_cv.best_params_)
print("accuracy :",logreg_cv.best_score_)

tuned hyperparameters :(best parameters) {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}
accuracy : 0.8464285714285713
```


TASK 5

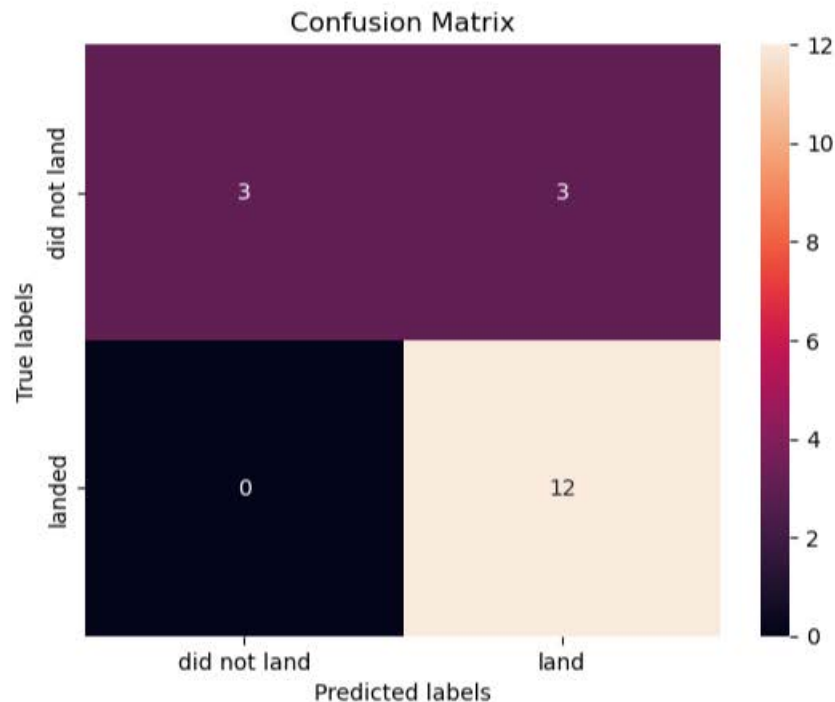
Calculate the accuracy on the test data using the method `score` :

```
[31]: logreg_score = logreg_cv.score(X_test, Y_test)
print("Logistic Regression Test Accuracy:", logreg_score)
```

Logistic Regression Test Accuracy: 0.8333333333333334

Lets look at the confusion matrix:

```
[32]: yhat=logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



Examining the confusion matrix, we see that logistic regression can distinguish between the different classes. We see that the problem is false positives.

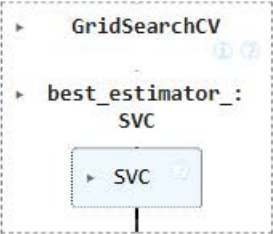
TASK 6

Create a support vector machine object then create a `GridSearchCV` object `svm_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
[33]: parameters = {'kernel':('linear', 'rbf', 'poly', 'rbf', 'sigmoid'),  
                  'C': np.logspace(-3, 3, 5),  
                  'gamma':np.logspace(-3, 3, 5)}  
  
svm = SVC()
```

```
[34]: parameters = {'kernel':('linear', 'rbf', 'poly', 'sigmoid'),  
                  'C': np.logspace(-3, 3, 5),  
                  'gamma': np.logspace(-3, 3, 5)}  
  
svm = SVC()  
svm_cv = GridSearchCV(svm, parameters, cv=10)  
svm_cv.fit(X_train, Y_train)
```

```
[34]:
```



```
  ▶ GridSearchCV  
    ▶ best_estimator_:  
      SVC  
        ▶ SVC
```

```
[35]: print("tuned hyperparameters :(best parameters) ",svm_cv.best_params_)  
      print("accuracy :",svm_cv.best_score_)
```

```
tuned hyperparameters :(best parameters) {'C': np.float64(1.0), 'gamma': np.float64(0.03162277660168379), 'kernel': 'sigmoid'}  
accuracy : 0.8482142857142856
```

TASK 7

Calculate the accuracy on the test data using the method `score` :

```
[36]: svm_score = svm_cv.score(X_test, Y_test)
      print("SVM Test Accuracy:", svm_score)
```

SVM Test Accuracy: 0.8333333333333334

We can plot the confusion matrix

```
[ ]: yhat=svm_cv.predict(X_test)
     plot_confusion_matrix(Y_test,yhat)
```

TASK 8

Create a decision tree classifier object then create a `GridSearchCV` object `tree_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

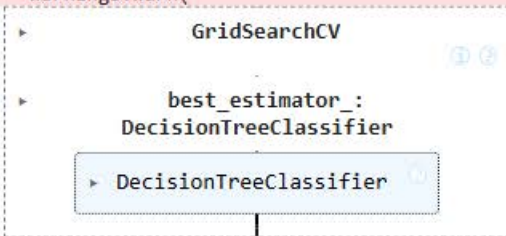
```
[ ]: parameters = {'criterion': ['gini', 'entropy'],
                  'splitter': ['best', 'random'],
                  'max_depth': [2*n for n in range(1,10)],
                  'max_features': ['auto', 'sqrt'],
                  'min_samples_leaf': [1, 2, 4],
                  'min_samples_split': [2, 5, 10]}
```

```
tree = DecisionTreeClassifier()
```

```
[37]: parameters = {'criterion': ['gini', 'entropy'],
                  'splitter': ['best', 'random'],
                  'max_depth': [2*n for n in range(1,10)],
                  'max_features': ['auto', 'sqrt'],
                  'min_samples_leaf': [1, 2, 4],
                  'min_samples_split': [2, 5, 10]}
```

```
tree = DecisionTreeClassifier()
```

[37]:



[38]:

```
print("tuned hyperparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)
```

```
tuned hyperparameters :(best parameters) {'criterion': 'entropy', 'max_depth': 2, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 10, 'splitter': 'random'}
accuracy : 0.8732142857142857
```

TASK 9

Calculate the accuracy of tree_cv on the test data using the method `score` :

[39]:

```
tree_score = tree_cv.score(X_test, Y_test)
print("Decision Tree Test Accuracy:", tree_score)
```

```
Decision Tree Test Accuracy: 0.6666666666666666
```

We can plot the confusion matrix

[40]:

```
yhat = tree_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```

TASK 9

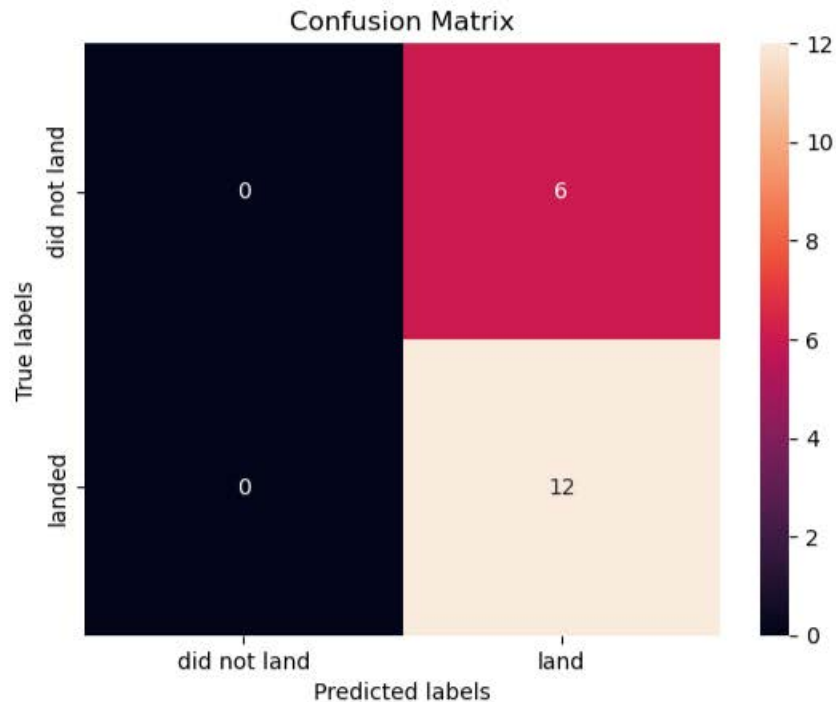
Calculate the accuracy of tree_cv on the test data using the method `score` :

```
[39]: tree_score = tree_cv.score(X_test, Y_test)
      print("Decision Tree Test Accuracy:", tree_score)
```

Decision Tree Test Accuracy: 0.6666666666666666

We can plot the confusion matrix

```
[40]: yhat = tree_cv.predict(X_test)
      plot_confusion_matrix(Y_test, yhat)
```



TASK 10

Create a k nearest neighbors object then create a `GridSearchCV` object `knn_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

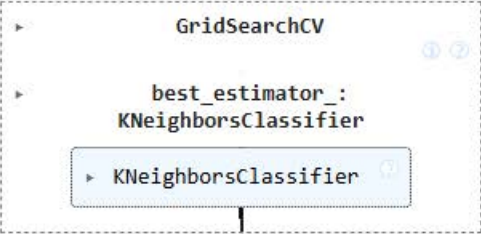
```
[ ]: parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
                  'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
                  'p': [1,2]}

KNN = KNeighborsClassifier()
```

```
[41]: parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
                  'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
                  'p': [1,2]}

KNN = KNeighborsClassifier()
knn_cv = GridSearchCV(KNN, parameters, cv=10)
knn_cv.fit(X_train, Y_train)
```

```
[41]:
```



```
GridSearchCV
├── best_estimator_: KNeighborsClassifier
│   └── KNeighborsClassifier
```

```
[42]: print("tuned hyperparameters :(best parameters) ",knn_cv.best_params_)
print("accuracy :",knn_cv.best_score_)
```

```
tuned hyperparameters :(best parameters) {'algorithm': 'auto', 'n_neighbors': 10, 'p': 1}
accuracy : 0.8482142857142858
```

TASK 11

Calculate the accuracy of `knn_cv` on the test data using the method `score`:

```
[ ]:
```

We can plot the confusion matrix

TASK 11

Calculate the accuracy of knn_cv on the test data using the method `score` :

[]:

We can plot the confusion matrix

[]:

```
yhat = knn_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```

TASK 12

Find the method performs best:

[]: